PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author
ITMO University, St. Petersburg, Russia
{author1, author2, author3}@itmo.ru

***Abstract*—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row after the row appears. We report violations of this requirement in five sources that we did not construct: a widely used feature-engineering library called without a cutoff-time table, as in its introductory example; the reference code written by the authors of this paper; code generated by two language models under a neutral prompt (53% and 35% of programs that ran); published expert SQL, in 8 of the 14 files of a benchmark's own user study that execute; and SQL written by an LLM agent working a benchmark task, in 2 of 34 queries. Inflation ranges from -0.3 to 16 AUC points and depends on the task more than on the defect: the largest violation we found ($345{,}938$ diverging cells) costs nothing measurable, and whether a one-character boundary error matters is decided by timestamp granularity, not by the code. We demonstrate PITFALL, which runs a feature program twice, on the full database and on a database truncated at the seed time, and reports any divergence. The check does not parse the program, has no false positives by construction, names the offending columns in 0.2–7.6 s and localises the leak by truncating one availability channel at a time, and scales to corpora: 37 agent-written queries receive verdicts in 19 minutes, where a static scan gave 5 candidates of which 3 are false. The univariate probe used by commercial AutoML platforms misses every source at DataRobot's default warning threshold, and H2O's more sensitive per-feature notification also fires on a correct pipeline.**

***Index Terms*—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code**

I. Introduction

In multi-table (relational) machine learning a prediction task is given as a table of triples *(entity, seed time, label)* over a database with foreign keys and timestamps [8]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows. Every aggregate may only read facts that existed at the seed time of the row it is computed for.

Leakage is a documented driver of irreproducibility in applied ML [1], [2], and this requirement is easy to violate: truncation is per row, so cutting the database once at the start of the test period says nothing about individual seed times; the requirement must hold along every join path, where the governing timestamp often belongs to a neighbouring table; and a row can carry columns written later than the row itself, such as a review attached to an order, so filtering by the order timestamp admits the row together with its future columns. This third case is the one most often missed.

Existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker) use the same univariate probe: fit a model on each single feature and flag it above a threshold, Gini Norm 0.85 (AUC 0.925) for DataRobot and AUC 0.80 for H2O [6], [7]. Static analyzers for notebooks [3] and pipeline instrumentation such as `mlinspect` [4] target preprocessing order and train/test contamination, not temporal leakage; asking a language model directly reaches 0.19 precision at 0.52 recall [5].

Contributions. (i) A simple execution-based check for feature programs, in the spirit of metamorphic testing: sound, one-sided, language-agnostic, extended with per-column availability and leak localisation by per-channel truncation. (ii) Evidence from five independent sources on two public databases (Section IV-C), including published expert SQL run under the check, and a demonstration that timestamp granularity rather than the code decides whether a boundary error costs anything. (iii) An execution-verified measurement of the rate at which two language models produce temporally incorrect feature code, by task construction and prompt guard. (iv) A working system and an interactive demonstration in which the checker fires where the industrial probe is silent, stays silent on correct code, and localises the leak.

II. Point-in-Time Correctness

A. Definition

Let D be a database, t a seed time and φ a feature program. Each row r has a row timestamp $\mathrm{time}(r)$; a column c of r may in addition have its own availability timestamp $\mathrm{avail}_c(r) \geq \mathrm{time}(r)$. Write $D|_t$ for the database in which rows with $\mathrm{time}(r) > t$ are removed and values with $\mathrm{avail}_c(r) > t$ are replaced by NULL. Then φ is *point-in-time correct at* t iff

$$\varphi(D, t) = \varphi(D|_t, t). \tag{1}$$

For a cutoff shift $\delta \geq 0$, $I(\delta)$ is the AUC of a *fixed* learner on features computed with cutoff $t + \delta$ minus its AUC at $\delta = 0$; the learner is fixed because in one published trajectory [9] swapping the boosting implementation on identical features moved AUC by 11.5 points.

B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t; co-emission and group leakage are other masks over the same machinery. Availability is defined per column: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (maximum 147) after the order row it is attached to. Some columns have no availability timestamp at all, e.g. a mutable status field kept without history; the truncated database is then identical to the full one and no execution-based check can decide them (we exclude `order_status` from all conditions). The class is easy to under-count: a negative control revealed a second instance, the derived `late` flag of never-delivered orders, which reads "not late" rather than "not yet known".

Why univariate probes miss this. A leaked signal spread over d aggregates has its visibility to any single-feature statistic fall with d while its effect on a multivariate model does not; concentrated in one feature, it is detected.

C. Differential execution and localisation

The check follows from (1): run φ twice and compare:

```
program(db,t,ents) !=
program(db.truncate(t),t,ents)
```

A divergence proves violation: the same program, given strictly less information, produced a different answer, whether through an aggregate, a join, or a statistic fitted on the full table; there are no false positives by construction. The guarantee is one-sided (a violation that does not manifest at the tested seed time is not observed) and the program must be deterministic.

The same primitive localises the leak. The database has a finite set of *availability channels*: rows of table T appearing after t, and values of column c becoming known after t. Truncating one channel at a time identifies the channels on which the output changes and the output columns each affects (seven runs in our database); masking exactly those channels and re-checking confirms that no other channel contributes. The mask is a diagnostic, not a fix, since any program passes on a pre-truncated input.

Why check rather than always truncate? With one seed time per call, truncating the input is itself a fix; but in training tables each row has its own seed time, so truncation must happen per row inside the program, which is what the check verifies, and a defect in code that will be reused should be reported, not masked once.

III. System

PITFALL sits between any feature builder and any tabular AutoML. SCOUT holds the availability map: which column governs the availability of which field, and which fields have none. Today it is a small hand-written declaration, which an LLM may propose from the schema, a question with an independently checkable answer, unlike "is there leakage here" [5]. BUILDER is any producer of a feature program: a library, an agent, or a person. GUARD runs the differential check and returns the verdict with the diverging columns and cells; on a violation LOCATOR runs the per-channel truncation. Clean programs are flattened and handed to an unmodified tabular AutoML such as AutoGluon [12]. The core is ∼170 lines of Python over pandas, independent of the builder's language.

Validation. On six reference programs with known verdicts at three seed times the checker was right on 18 of 18 combinations. A negative control with the seed time set past every timestamp caught three defects of our own before the measurements that depended on them and surfaced the second undecidable class of Section II-B. The third is worth naming: on rel-amazon the control fired on the same files the checker was calling violations, because truncation changes the query plan and floating-point summation is not associative. Resetting the tolerance from the control's own noise floor (1.7×10^{-11} relative) makes all four files clean. Four leak verdicts were false, and no reading of the checked code would have found it.

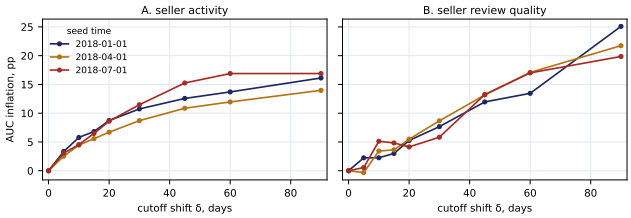

Fig. 1. Inflation $I(\delta)$ as the cutoff moves δ days past the seed time (Olist). A one-month error costs 5.8–11.5 AUC points; no cutoff at all, up to +27.0 (A) and +18.0 (B).

IV. Evidence

Setup. Two databases. Olist [13], a public Brazilian marketplace database: 7 tables, 112,650 order line items, 2016–2018; three tasks in RelBench [8] form (seller activity, A; seller review quality, B; product demand, C), three seed times in 2018, 90-day horizon, 750–870 sellers and 4,500–5,800 products per seed time. rel-f1 [8], the Ergast Formula 1 database: 9 tables, 26,080 driver-race results, 1950–2023; task *will this driver retire from the next race*, seed time at midnight of race day, three test seasons of 1,042–1,203 rows, ten preceding seasons for training. One LightGBM configuration with a fixed seed everywhere. Intervals are paired percentile bootstraps of the AUC *difference* on the same test rows (2,000 resamples).

A. The cost of a shifted cutoff

$I(\delta)$ increases monotonically over $\delta \in [0, 90]$ days on Olist tasks A and B at all three seed times (Fig. 1). On task C, controlling the cutoff separately for the entity's own history and for aggregates reached through a join to the seller, leaking only through the join costs +3.0 [+2.2, +3.9] to +5.1 [+4.0, +6.1] points and is invisible to the probe (Section IV-D).

B. When an off-by-one costs everything and when it costs nothing

Whether a boundary error matters is decided by the timestamps, not by the code, and rel-f1 settles this inside one database: race stamps carry no time of day before 2005 and carry the start time from 2005 on. An outcome is available strictly after the race starts, so the inclusive cutoff $\leq t$ admits the race being predicted in the first era and nothing in the second. The checker separates the eras before any model is fitted, in 0.02 s per seed time: 65–68 diverging cells on 1996, 2000 and 2004, clean on 2010 and 2015. The same one-character error then costs -0.45, $+2.45$ and $+3.27$ [$+1.20$, $+5.50$] points on the three day-stamped cells and exactly 0.00 on all three second-stamped cells. On Olist, where events carry seconds, this error also cost 0.00; without rel-f1 we could not have told whether that generalises.

Leaking only through the join path replicates at $+0.72$ to $+1.49$ points while the probe moves by at most 0.009; removing the cutoff altogether costs -1.9 to -7.0 points here, with no upper bound, because aggregating a driver's whole career destroys the recency structure the task relies on.

C. Sources of violation

Five sources, none constructed by us; the single-feature probe stays below DataRobot's warning threshold on every one.

A library default. `featuretools` [10] 1.31.0 with `ft.dfs` and no cutoff-time table, the form of its introductory example [11], supports cutoff times but does not indicate that they are needed: 11 columns and 19,334 cells diverge at all three seed times, worth 14.8–16.3 points.

Our own reference code. It filtered history by order time and took all columns of the admitted rows, two of which carry their own later timestamps (the review score and the delivery outcome); the checker flagged exactly the four review and delivery aggregates ($\sim$2,000 cells) and no other column, worth 0.2 points on task A and 3.1–5.3 on task B.

Published expert SQL, executed. In the hand-written feature SQL of the RelBench user study [8] (15 files, 172 joins) a manual audit had found two violations, with six of eight scanner candidates false. Each file is a DBT/Jinja template over DuckDB; the availability relation comes from RelBench's own `time_col` declarations and truncation is a predicate pushed into the Parquet scan. Fourteen files execute; **8 diverge** and 6 are clean at every seed time, the latter under truncations removing 2.7×10^5 to 2.1×10^7 rows. The three rel-f1 files share one cause, a block reading `races` rows dated after the seed time; on `stack/user-badge` it is a global frequency over `badges` with no cutoff (the violation the audit did find), where the checker names eight affected columns against the audit's two; on `stack/post-votes`, which the audit cleared, the output depends on 2,730 `users` rows dated after the posts they own. The file that does not execute reads `posts.AcceptedAnswerId`, a mutable field with no history that RelBench itself deletes with the comment *remove time leakage columns* — an independent party reaching our Section II-B verdict on the same column.

How much a clean verdict is worth. The three rel-event files join `users` without a cutoff, so six demographic columns read rows of users who registered after the seed time. Over five seed times per file the oracle returns 12 leak verdicts and 3 clean, and all three clean verdicts fall on one date. Counting directly, the violation reaches 87.5% of label rows and is present at 23 of the task's 24 seed times — absent at exactly one, 2012-11-29, RelBench's own test seed time. Training labels are contaminated almost everywhere and test labels are clean: no inflated test score, but train and test built by different rules. The date is not a coincidence. The latest registration among *labelled* users is 2012-11-28, because a user joining in the last two weeks has no future window left in the data and receives no label; the last seed time therefore lies past every labelled registration. Data beyond a seed time decreases monotonically in that seed time, which makes a benchmark's test seed the least sensitive one available for this check — the validation seed already retains only 52 affected rows. Seed times must come from the middle of the training split, not from the split the benchmark scores on.

An agent-written corpus. The 37 distinct queries an LLM agent wrote across 32 trials on `stack/user-engagement` had only a static scan: 5 candidates, no executed verdict. Running them takes 19 minutes: 34 execute, of which **2 diverge** and 32 are clean; 2 build a Cartesian product of three histories per user and exceed a 300 s budget, 1 names a table that does not exist. Both violations are the same missing cutoff on the join to `users`, giving account ages down to $-2{,}021$ days on 57 of 25,233 label rows; the query carrying it appears in 31 of the 32 trials. Against execution the scan errs both ways: 3 of its 5 candidates are false and 2 real, correcting our own earlier reading of that corpus.

D. What the industrial probe reports

Across every run, no value of the probe separates leaking runs from clean ones. The clearest case is leakage through a join: the probe returns $0.714/0.722/0.717$ under a correct cutoff and the same values under leakage, while the model gains 3.0–5.1 points. On task B no condition, including the complete absence of a cutoff ($+18.0$ points), reaches the DataRobot threshold, and only shifts of two months or more reach H2O's 0.80; the sensitive threshold errs the other way, as the correct task-A pipeline scores $0.837/0.861/0.833$, above 0.80 at every seed time, because recency is a legitimate predictor of churn. Both errors replicate on rel-f1 (maximum probe 0.820, movement at most 0.015 between correct and violating, H2O firing on the correct pipeline as often as on the violating one), and on the executed expert SQL the probe is identical to six significant figures in both regimes.

E. Generated feature code

We asked two code models — `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code` — for a Python function computing features for task C, giving the schema

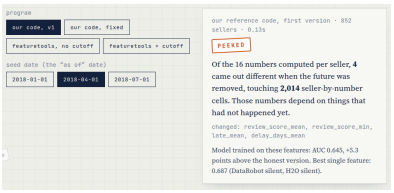

Fig. 2. The demonstration page: program and seed-date selectors, and the verdict with a plain-language summary. Below it, not shown, the outputs side by side.

and the seed-time column and nothing else; the words "leakage" and "point-in-time" do not appear. Verdicts come from the checker; five flagged programs were read by hand and confirmed. Under the neutral prompt 53.3% and 35.3% of the programs that ran were incorrect (30 and 17 of 40 ran; failures to execute are excluded, which may bias the rate either way). The failure mode is the same in every case examined: the model filters the entity's own history by the primary event timestamp and ignores the later timestamp of the joined entity. We make no claim about models in general. Task construction is decisive: on a second task with a single seed time per entity and no secondary time axis, both models produce 0 violations in 24 and 28 running programs. Prompt guards help partially but not to zero: a reminder and then a per-aggregate requirement cut the pooled rate from 46.8% to 14.6%.

F. Cost of a violation is task-dependent

The same defect in the same code costs 3.09 to 5.26 points on task B, while on task A every interval contains zero (0.16 to 0.32); among model-generated programs the median inflation is 0.11 points. The expert SQL makes the same point on code we did not write: rebuilding its features on a per-row truncated database and refitting the same model, the violations cost -0.33 and $+0.95$ points on the two rel-f1 tasks and -0.02 on `stack/user-badge`, the last over $345{,}938$ diverging cells in $255{,}360$ test rows. Neither the fact of a violation nor its size predicts its cost. Correctness therefore has to be tested separately from the quality metric, which an execution-based checker does and a metric-based heuristic cannot.

V. Demonstration

The demonstration is a self-contained interactive page built from live runs (Fig. 2) plus a console runner, `python3 demo.py`. A visitor drags the seed date over one seller's real orders and watches a naïve "average review" feature change; compares programs and seed dates side by side; plays "fool the checker"; moves the cutoff and the probe threshold; and submits a feature function of their own. The console scenes replay Section IV-C live: `ft.dfs` without cutoffs, VIOLATION in 7 s; our own code, VIOLATION in 0.2 s naming the four review and delivery aggregates while both probe thresholds stay silent; the repaired program, CLEAN; and per-channel localisation tracing each diverging column to the timestamp it enters through.

VI. Limitations

(i) One-sided. The check proves violation, not correctness, and only with respect to the observed database, the chosen mask and the seed times tested; the rel-event case above shows that gap is not hypothetical. *(ii) Undecidable columns.* Mutable fields kept without history cannot be checked by execution. *(iii) The availability map.* A wrong timestamp means the wrong thing is checked, and the map is a modelling decision: RelBench's `time_col` places the rel-f1 race calendar after the seed time, though a practitioner may consider it published in advance; the check decides code against a declared relation, not against intent. *(iv) Scope.* Two databases, four tasks; the expert-SQL corpus executes on 14 of 15 files, and audit and checker disagree on 7 of those 14, always with the checker finding more. *(v) Determinism.* Columns that differ between identical reruns carry no verdict. *(vi) Final-program metric undercounts:* validation-driven selection can discard a leaking intermediate trial (observed once).

VII. Conclusion

Point-in-time violations occur in library defaults, in published expert code and in generated code, and the univariate probe used to catch them misses the regimes reported here. Their cost is not predictable from their existence or size, so correctness must be tested rather than scored; differential execution decides it in seconds and localises the leak with the same primitive. A clean verdict is only as strong as the seed times it was taken at.

Data, Ethics and Availability

Experiments use public datasets: Olist (Kaggle, CC BY-NC-SA 4.0, anonymised by its publisher) and the rel-f1/rel-stack databases and expert SQL released with RelBench [8]; no human subjects. Language-model outputs are released unedited. Code, data and all reported numbers: https://github.com/stas1f1/pitfall.

References

[1] S. Kaufman, S. Rosset, and C. Perlich, "Leakage in data mining: formulation, detection, and avoidance," in *Proc. KDD*, 2011.
[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.
[3] C. Yang *et al.*, "Data leakage in notebooks," in *Proc. ASE*, 2022.
[4] S. Grafberger *et al.*, "mlinspect: a data distribution debugger for ML pipelines," in *Proc. SIGMOD*, 2021.
[5] N. Alturayeif and J. Hassine, "Data leakage detection in ML code," *PeerJ Computer Science*, 11:e2730, 2025.
[6] DataRobot documentation, "Data quality assessment: target leakage," docs.datarobot.com, 2026.
[7] H2O.ai, Driverless AI 1.11 User Guide, "Data leakage and shift detection," docs.h2o.ai, 2026.
[8] J. Robinson *et al.*, "RelBench: a benchmark for deep learning on relational databases," *NeurIPS D&B*, 2024.
[9] "RelAgent: agentic feature engineering for relational data," arXiv:2605.07840, 2026.
[10] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis," in *Proc. DSAA*, 2015.
[11] Featuretools 1.31 documentation, "Handling time," featuretools.alteryx.com, 2026.
[12] N. Erickson *et al.*, "AutoGluon-Tabular," arXiv:2003.06505, 2020.
[13] Olist, "Brazilian e-commerce public dataset," Kaggle, 2018.